LOW LEVEL DESIGN

The LLD Runbook

Ten problems, worked the way you would actually solve them.

Part 1 gave you the vocabulary. This is where it earns its keep. Each problem runs end to end in two pages — the questions worth asking, the model that follows, and the follow-up the interviewer is waiting to spring.

Attempt each one before reading the answer. The value is not in the solutions; it is in finding out where your instincts differ from them.

Varun Kashyap

How to read these problems

Every problem takes exactly two pages and the two pages always do the same two jobs. Once you know the shape, you can find any part of any problem without hunting.

LEFT PAGE — THE DESIGN

- **Problem statement** as an interviewer would give it, ambiguity intact.
- **Clarifying questions** worth asking, and which ones actually change the design.
- **Requirements**, functional and non-functional, separated.
- **Entities** — the nouns that survive scrutiny.
- **Class diagram** of the model those nouns produce.

RIGHT PAGE — THE DEPTH

- **Relationships** and the **patterns** that fell out of them.
- **Code sketch** of the decisive part, with full source linked.
- **Complexity**, then the **follow-ups** to expect.
- **Alternative design** — the other defensible answer.
- **Common mistakes**, and what the interviewer is really watching.

How to actually use them

Read the problem statement and the clarifying questions, then stop. Give yourself fifteen minutes and a blank page before you read another word. What you are training is the sequence from **6.1**, and you cannot train it by reading someone else's finished answer.

When you do read on, compare structure rather than names. If your classes differ from mine but your seam is in the same place, you have the right answer. If your seam is somewhere else, work out which of us guessed the axis better — sometimes it will be you.

> EVERY PROBLEM ENDS WITH THE FOLLOW-UP
>
> That is deliberate, and it mirrors the round itself. The design is the setup; the change in requirements is where the marks are. If you only have time to study half of each problem, study the right-hand page.

P1 LRU Cache

PROBLEM STATEMENT

"We show a user's saved delivery address on the checkout page, and it needs to load in near real time. The user base is large. We care most about the people who come back often — for everyone else, being a little slower is acceptable."

That is close to how it was actually put to me. Notice that the words *cache* and *least recently used* never appear. Hearing them anyway is the first half of the test.

CLARIFYING QUESTIONS

- Am I right that this is a bounded in-memory cache keyed by user? *Say your interpretation out loud before you design to it.*
- Does `get` count as a use? *Yes — and if you don't ask, ask yourself why you assumed it.*
- Single-threaded? Say you're assuming so, and that you'll revisit it.
- Any expiry or time-to-live? Usually out of scope, but worth ten seconds to confirm.

FUNCTIONAL

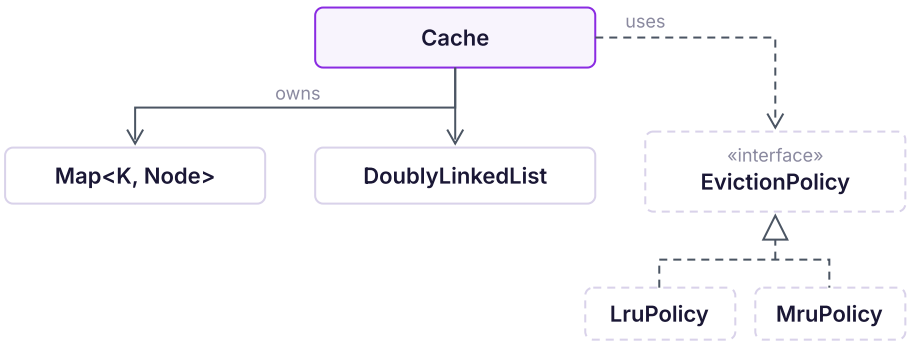
- `get(key)` returns the value and marks it most recently used.
- `put(key, value)` inserts or updates, and marks it most recently used.
- At capacity, `put` evicts the least recently used entry first.

NON-FUNCTIONAL

- Both operations in O(1) — this is the whole point of the problem.
- Memory bounded by capacity, not by total keys ever seen.
- Eviction order must be exact, not approximate.

ENTITIES

`Node` holding key, value and both links · `DoublyLinkedList` maintaining recency order · a map from key to node for constant-time lookup · `Cache` coordinating the two · `EvictionPolicy`, the seam.



The map gives O(1) lookup; the list gives O(1) reordering. Everything dashed is what the follow-up added — my first version decided eviction inside `Cache`.

COMPLEXITY

`get` O(1) · `put` O(1) · space O(capacity). The map buys lookup; the doubly linked list buys removal without a scan.

ALTERNATIVE DESIGN

`LinkedHashMap` in access order does this in five lines — but it hard-codes LRU, which is exactly what the follow-up attacks.

P1 LRU Cache, in depth

RELATIONSHIPS

- `Cache` owns the map and the list outright.
- The policy is handed in, not created — that is what makes it swappable.

PATTERNS USED

- **Strategy** for eviction — the only pattern this problem needs.
- A **Factory** only if policies arrive from config.

WHAT THE FOLLOW-UP ACTUALLY CHANGED

FIRST VERSION — CORRECT, AND THE RULE IS BAKED IN

```
Node victim = order.tail();              // "least recently used", decided right here
map.remove(victim.key);
order.remove(victim);
```

AFTER THE FOLLOW-UP — THE RULE BECOMES A COLLABORATOR

```
void put(K key, V value) {
    if (map.size() == capacity) {
        Node victim = policy.select(order);  // the only line that changed
        map.remove(victim.key);
        order.remove(victim);
    }
    map.put(key, order.addFront(new Node(key, value)));
}
```

One line at the call site, one interface with one method, and a constructor that accepts it. Update path omitted for clarity.
`github.com/varunkashyap-vkd/lld-runbook/lru-cache`

FOLLOW-UPS TO EXPECT

- **"Make it most recently used instead."** The one this problem exists for. Behind an interface it is a new class; without one, a rewrite.
- **"Now make it LFU."** Say plainly that the interface holds but the structure behind it does not.

COMMON MISTAKES

A singly linked list, so removal becomes O(n) and the point is lost · forgetting that `get` reorders, the most common correctness bug here · no key inside the node, so eviction cannot clean up the map.

INTERVIEWER'S PERSPECTIVE · THE ROUND I NEARLY LOST

I was told to expect an LLD round, then the interviewer arrived with a live code editor — so I quietly decided it was really a DSA round. I recognised the shape, wrote a working cache well inside the time, and we tested it together. It was correct.

Then he moved the requirement: what if product wants this faster for the people who use us *least*? Eviction was hard-coded inside my cache class — there was nothing to swap. What saved me was having twenty-five minutes left. I said so, took two minutes, and came back with the policy behind an interface. When he asked what a third policy would cost, the answer was one class and about five lines. The feedback was about adapting to change, not about caches.

If you take one thing from this problem: it is only easy until the requirement moves. In an LLD round, write the first version as though the rule will change — because it will.